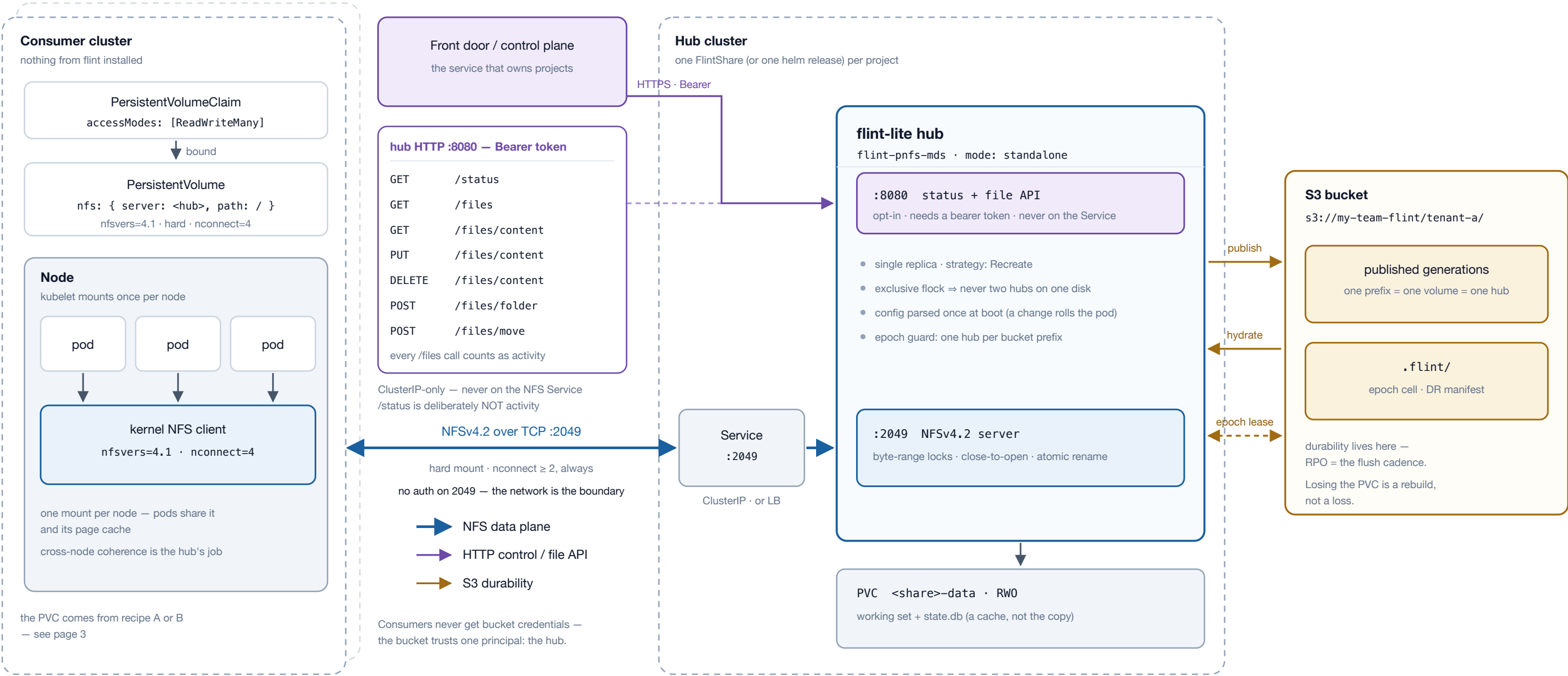
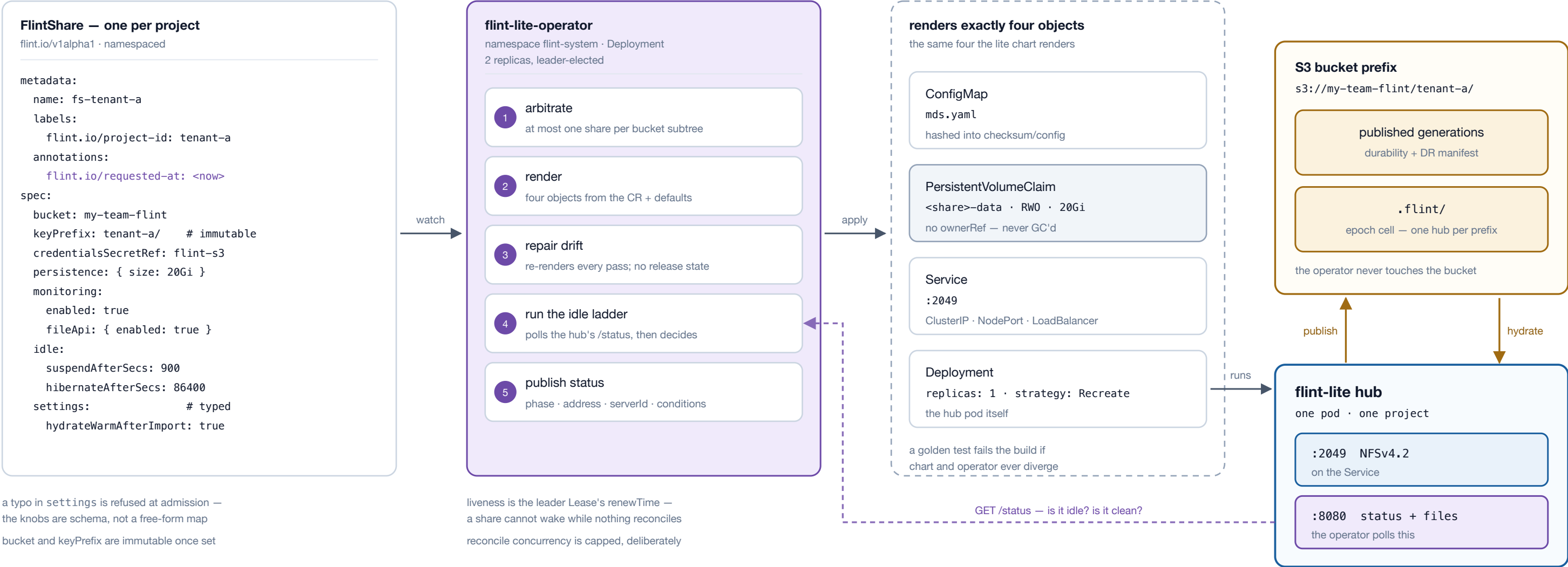


A single pod serves real POSIX over NFSv4.2. The PVC is the working set, an S3 prefix is the durability, and a second listener exposes the same filesystem over HTTP for callers that cannot mount.



One hub is the coherence authority for one tree — two hubs over one prefix is split-brain, which the store-side epoch lease fences and the operator refuses up front. **nconnect ≥ 2 is not optional:** without it the kernel opens exactly one connection and silently refuses every additional trunk — no error, on either side, ever. **The two doors are deliberately separate:** :2049 is on the Service and may become a LoadBalancer; :8080 can rewrite every file in the share, so it stays ClusterIP-only and never joins it. Each HTTP endpoint is an NFS compound dispatched in-process, not a second reader of the directory — so it inherits eviction handling, the write gate, symlink rules and the tier's capture notes. A cold file read through either door hydrates from the bucket first: NFS parks the caller with NFS4ERR_DELAY, HTTP answers 503 + Retry-After. **They are secured differently, and only one of them has a credential.** The file API needs a bearer token per share and mounts no routes at all without one — 404, not 401. Port 2049 is AUTH_SYS: the client asserts its own uid and the server takes it, so POSIX modes are enforced against a self-declared identity and **reachability is the boundary.** That control is networkPolicy in both charts — off by default, admitting nothing when both client lists are empty, and silently ignored by a CNI that does not enforce it.

One custom resource per project replaces one helm release per project. The operator re-renders the same four objects from the CR on every pass — there is no reusable release state to go stale, and nothing it does ever reaches the bucket.



Three things it will not do

- 1 · It never touches a bucket. Deleting a share deletes Kubernetes objects; S3 is exactly as durable as before.
- 2 · It never garbage-collects a PVC — the claim carries no ownerReference, deliberately.
- 3 · It will not run two hubs on one bucket subtree.

The fleet is one command

```
$ kubectl get flintshares -A
```

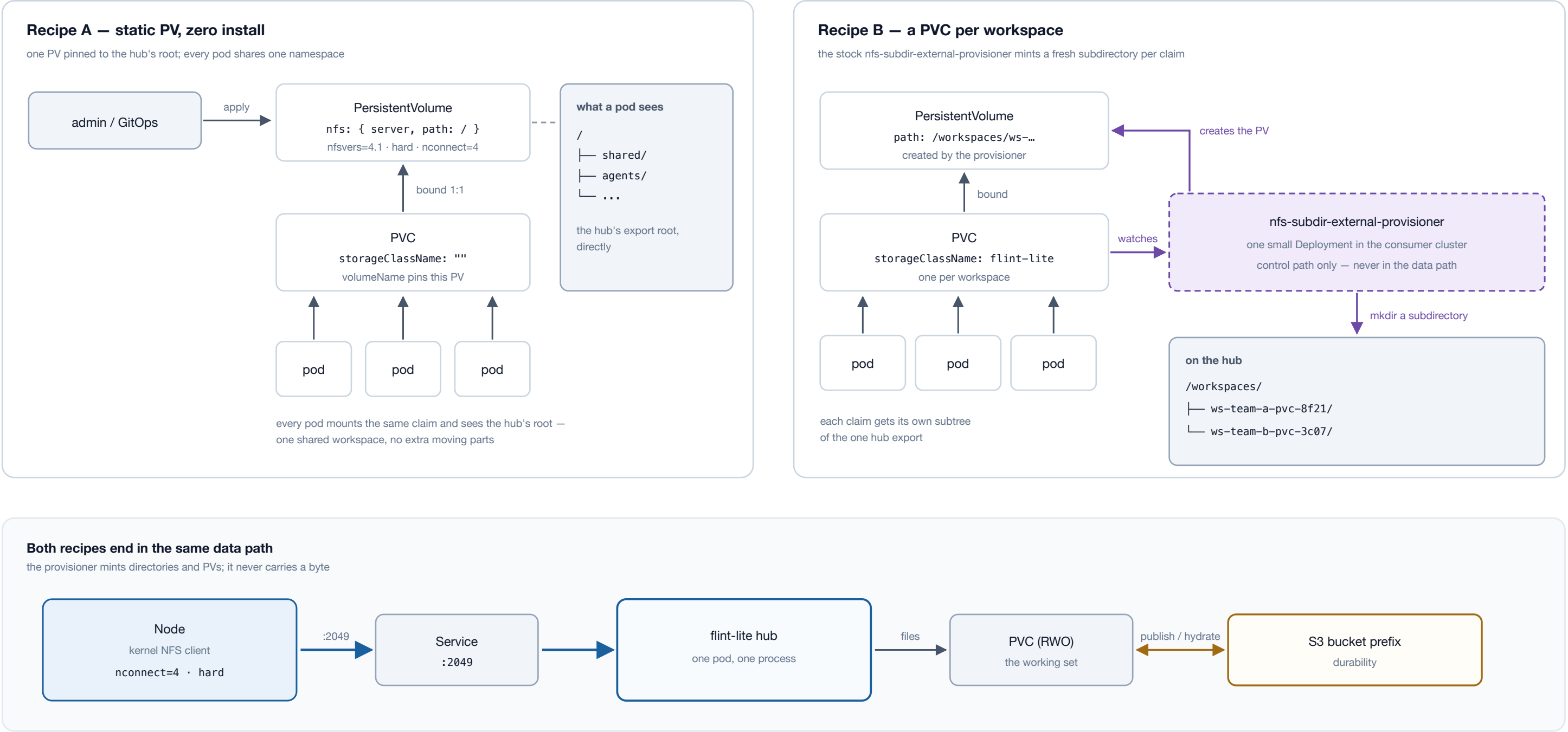
NAMESPACE	NAME	PHASE	ADDRESS
workspaces	tenant-a	Ready	tenant-a.workspaces.svc:2049
workspaces	tenant-b	Starting	
workspaces	tenant-c	IdleSuspended	

The front-door contract

```
GET flintshares/fs-<id> → 404? create
PATCH flint.io/requested-at: <now>
poll until .status.phase == Ready
read .status.address, .status.serverId
a changed serverId means a fresh PVC — remount
```

Derive the share's name from the project id (fs-<project-id>) and label it flint.io/project-id — that is what makes ensure-live idempotent: two front-door replicas racing a first touch issue the same create, one gets 409 AlreadyExists, and both proceed. **Uniqueness is enforced from the controller's cache**, not by the API server: Kubernetes cannot express "at most one share per (endpoint, bucket, prefix subtree)" in CEL, since both CEL and a ValidatingAdmissionPolicy see one object. Oldest wins; the loser goes Failed with a Conflict condition and gets no hub. **That enforcement stops at the cluster boundary.** Two clusters each admit a share on one prefix and neither sees the other — and the store-side epoch only catches it when the prefixes are equal, because tenant-a/ and tenant-a/sub/ mint different epoch objects and never contend. Own prefix allocation upstream of Kubernetes.

Consumers install nothing from flint either way: the data path is the node kernel's NFS client. The choice is only who mints the PV — you, once, by hand; or a stock provisioner, once per workspace.



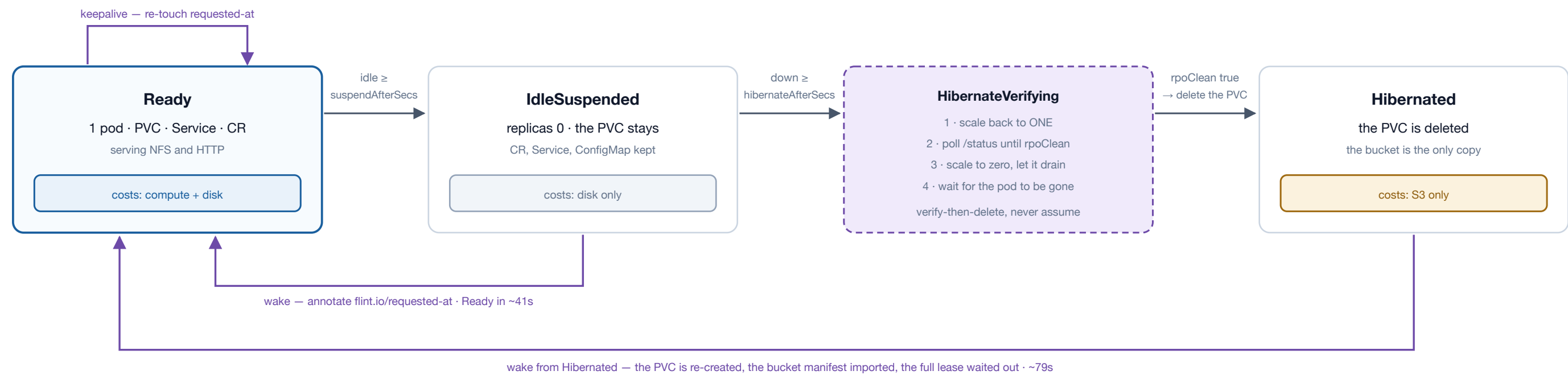
The same nconnect caveat applies to both recipes — recipe B passes mount options straight to the node's mount, so leaving it out gets one connection per node and no error saying so. **Recipe A is one shared tree**; recipe B is StorageClass semantics on top of it, so pick B when each workspace should be isolated and minted on demand, A when the point is that everybody sees the same files. Kubelet mounts once per node: pods co-scheduled on a node share one kernel client and one page cache, while cross-node and cross-cluster coherence — locks, close-to-open — is the hub's job. **Either way, a suspended hub is not woken by a mount.** A hard mount against a scaled-to-zero share hangs, because there are no endpoints and nothing to notice the attempt — which is why the wake annotation is written before an address is handed to anyone.

A tiered hub does real work before its listener binds, and the operator winds an unused share down in two rungs — each opt-in, each giving back something different. Nothing in the data path ever starts a hub.

Starting a hub — what happens before port 2049 opens



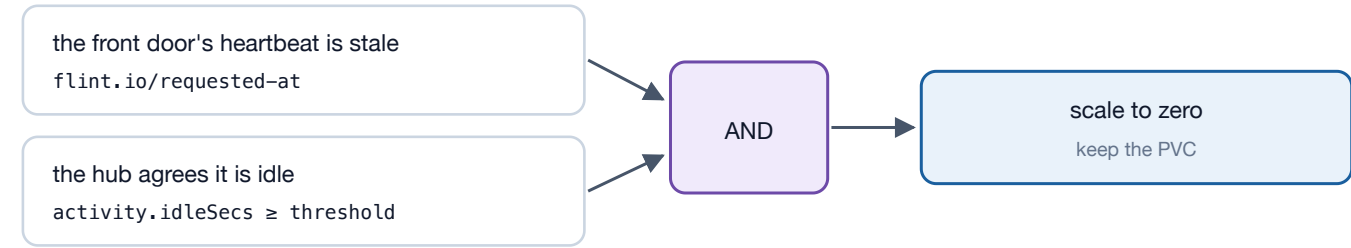
The idle ladder — each rung gives back something different



Nothing in the data path wakes a share: an NFS mount against a scaled-to-zero hub hangs, the file API answers 503 fast. Something must write the annotation — and a share cannot wake while no operator is reconciling.

spec.lifecycle: Suspended is an admin decision and a wake request does NOT override it — the phase reads Suspended, not IdleSuspended, so a front door can tell "will wake on request" from "never coming back".

Suspending needs two independent signals to agree



A hub that cannot be polled is never suspended — an unreachable hub is an unknown hub, not an idle one, and HubReachable says so.

What each rung still holds

	CR	Svc + CM	PVC	Pod
Ready	held	held	held	held
IdleSuspended	held	held	held	released
Hibernated	held	held	released	released

Deleting the CR drops every object — an archived project costs zero Kubernetes objects. The bucket is never touched, under any policy.

The two rungs are not one setting with two numbers. Suspend keeps the disk, so waking is a pod start and an epoch re-claim on the same `state.db` — safe for any share, tiered or not. Hibernate deletes the PVC, so it requires `spec.bucket` and waking is a full DR import. **Hibernation is verified at drain time, not assumed:** the hub exits 0 whether or not it flushed and scale-to-zero destroys the pod, so the operator scales back to one and waits for `rpoClean` before it deletes anything. `rpoClean: null` — a share with no bucket — is a refusal, not a pass. **Hibernated wakes are slower than they look** (~79s against ~13s for the same-identity case): destroying the volume destroys the `serverId`, so self-recognition cannot short-circuit the lease and the claim waits out the full `lease_misses × heartbeat`. Every long-parked project pays that on its first open. **Do not keepalive from a liveness poller.** `/status` is deliberately not activity and the file API deliberately is — a UI that refreshes a listing on a timer pins its project awake forever, and a conditional GET answering `304` pins it exactly as hard.